FLUTTER · DART · i18n

FLUTTER INTERNATIONALIZATION

Flutter ile Çok Dilli Uygulama Geliştirme

`Locale` `ARB` `gen-l10n` `intl`

Kaynak: Flutter resmi dokümantasyonu — docs.flutter.dev/ui/internationalization



Hazırlayanlar: Gizem Aksoy, Gaye Aksoy · Ders: Mobil Programlama I

Internationalization (i18n) Nedir?



Tanım

Uygulamayı; kaynak kodunu yeniden yazmadan farklı dillere ve bölgelere uyarlanabilecek şekilde tasarlama sürecidir. Kısaca: uygulamanın çok dile hazır altyapısı.

i18n kısaltması nereden geliyor?

i n t e r n a t i o n a l i z a t i o n

i + 18 harf + n → **i18n**

i18n Neden Kullanılır?



Metinler koddan ayrılır

Ekrandaki her metin kodda sabit yazılmaz; bir anahtar (key) ile çağrılır.

Biçimler bölgeye bırakılır

Tarih, sayı ve para biçimleri kullanıcının bölgesine göre ayarlanır.

Metin yönü desteklenir

Soldan sağa (LTR) ve sağdan sola (RTL) yazılan diller aynı kodla çalışır.

Gerçek hayattan örnek: e-ticaret uygulaması

× Sabit metin

```
Text('Sepete ekle')
```

✓ Uluslararasılaştırılmış

```
Text(l10n.addToCart)
```

`addToCart` → app_tr.arb → "Sepete ekle"

`addToCart` → app_en.arb → "Add to cart"

Yeni dil = yeni çeviri dosyası; kod değişmez.

Localization (l10n) Nedir?

l o c a l i z a t i o n

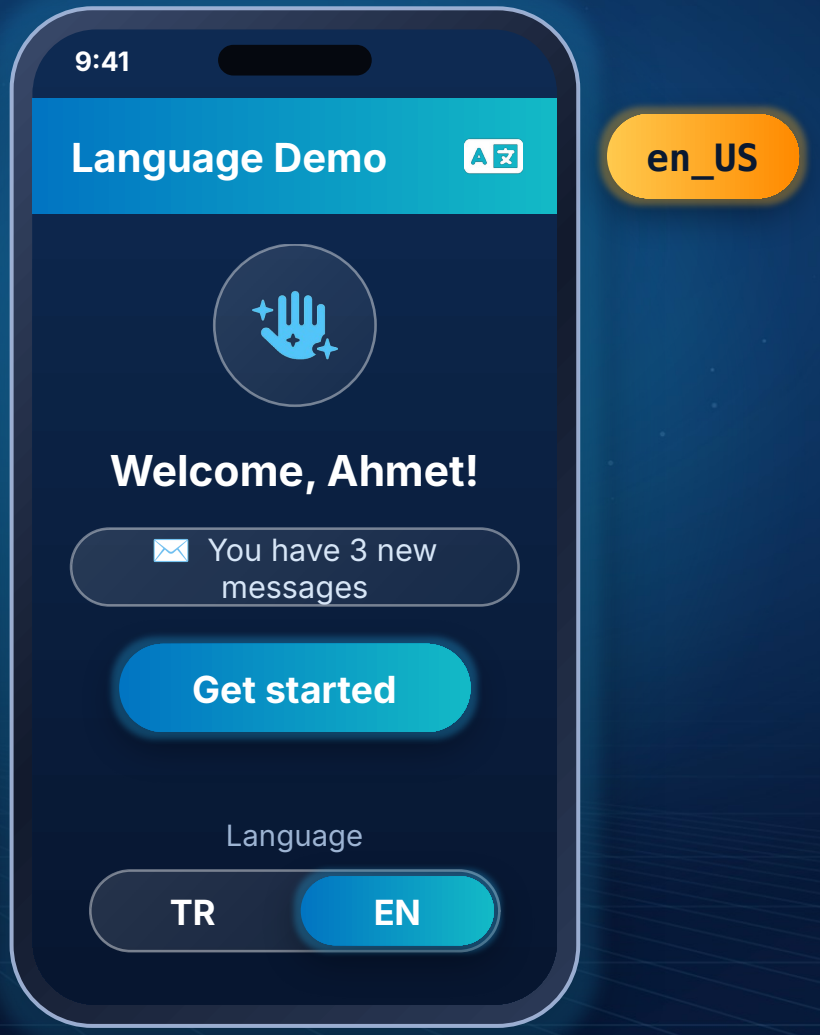
l + 10 harf + n → **l10n**

Tanım

Uluslararasılaştırılmış bir uygulamayı belirli bir dil ve bölgeye uyarlamaktır: çeviriler, tarih/sayı/para biçimleri ve kültürel uyum.

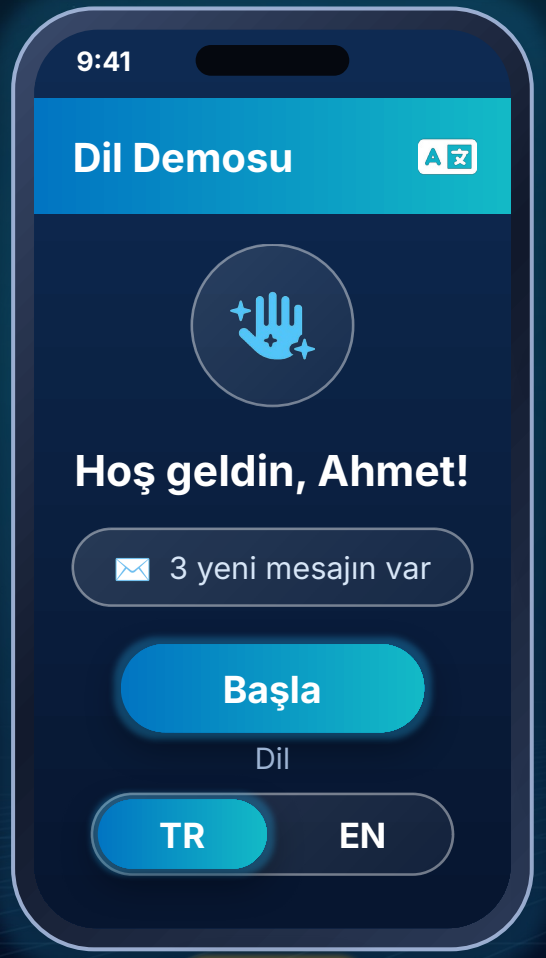
i18n ile ilişkisi

Önce i18n ile altyapı kurulur, sonra her dil için l10n yapılır. Sağdaki İngilizce ekran, bir sonraki slaytta Türkçeye yerelleşiyor.



örnek arayüz

i18n ve l10n: Altyapı ve İçerik



tr_TR örnek arayüz

	i18n · Uluslararasılaştırma	l10n · Yerelleştirme
Amaç	Uygulamayı çok dile hazır hâle getiren altyapı	Bu altyapıya belirli bir dilin içeriğini eklemek
Ne yapılır?	Metinleri koddan ayırmak, biçimlendirmeyi locale'e bırakmak	Çeviri yapmak; tarih, sayı ve para biçimlerini uyarlamak
Ne sıklıkla?	Genellikle bir kez, proje başında	Her yeni dil ve bölge için tekrar
Flutter'da	flutter_localizations, delegate'ler, gen-l10n kurulumu	app_en.arb, app_tr.arb … çeviri dosyaları

Çok Dilli Uygulamalar Neden Önemlidir?



Küresel kullanıcı kitlesi

Uygulama mağazaları dünyaya açık; kullanıcı kendi dilini görmek ister.

Kullanıcı deneyimi

Ana dilde arayüz daha kolay anlaşılır, güven verir, hataları azaltır.

Bölgesel farklılıklar

Tarih (27.10.2026 / 10/27/2026), sayı, para birimi ve yazı yönü değişir.

Gerçek örnekler

Telefonun dili değişince bankacılık, harita ve sosyal medya uygulamaları da değişir.

BÖLÜM

02

Flutter'da Uluslararasılaştırma



- Mimari
- Paketler
- Proje hazırlığı
- MaterialApp
- Locale

- `flutter_localizations`
- `intl`
- `gen-l10n`

Flutter Internationalization Mimarisi



Nasıl çalışır?

1. Cihaz dili, supportedLocales listesiyle eşleştirilir.
2. Her delegate, load(locale) ile o dile ait nesneyi üretir.
3. Locale değişince Localizations, bağlı widget'ları yeniden çizer (InheritedWidget mantığı).

Flutter Localization Paketleri ve Araçları

`flutter_localizations`
Flutter SDK ile gelir

- Material, Cupertino ve Widgets bileşenlerinin hazır çevirileri
- Global...Localizations delegate'leri
- Metin yönü (LTR/RTL) bilgisi

`intl`
Dart paketi (pub.dev)

- Mesaj biçimlendirme: plural ve select (ICU)
- DateFormat: tarih ve saat
- NumberFormat: sayı ve para birimi

`gen-l10n`
Flutter aracı (flutter gen-l10n)

- ARB dosyalarını okur
- Tip güvenli AppLocalizations sınıfı üretir
- generate: true ile otomatik çalışır

Birbirleriyle ilişkisi



Üretilen AppLocalizations kodu, çoğul/tarih/sayı biçimlendirmesi için intl paketini kullanır; flutter_localizations ise hazır widget metinlerini sağlar.

Proje Hazırlığı: pubspec.yaml

Terminal

```
$ flutter pub add flutter_localizations --sdk=flutter
$ flutter pub add intl:any
```

pubspec.yaml

```yaml
dependencies:
  flutter:
    sdk: flutter
  flutter_localizations:    # SDK ile gelir
    sdk: flutter
  intl: any                 # biçimlendirme

flutter:
  generate: true            # gen-l10n açık
  uses-material-design: true
```

1 flutter_localizations ekle

Material/Cupertino widget'larının çevirileri ve delegate'ler. Sürümü SDK ile aynıdır (sdk: flutter).

2 intl ekle

Tarih, sayı, çoğul ve seçim biçimlendirmesi için Dart paketi.

3 generate: true

flutter pub get / flutter run sırasında çeviri kodunun otomatik üretilmesini açar.

Bilgi: Flutter varsayılan olarak yalnızca ABD İngilizcesi (en_US) yerelleştirmesi sunar. Diğer diller bu kurulumla eklenir.

MaterialApp ve Localization Ayarları

main.dart

```dart
import 'package:flutter/material.dart';
import
'package:flutter_localizations/flutter_localizations.dart';
import 'l10n/app_localizations.dart';

MaterialApp(
  title: 'i18n Demo',
  localizationsDelegates: const [
    AppLocalizations.delegate, // bizim metinler
    GlobalMaterialLocalizations.delegate,
    GlobalWidgetsLocalizations.delegate,
    GlobalCupertinoLocalizations.delegate,
  ],
  supportedLocales: const [
    Locale('en'), // İngilizce
    Locale('tr'), // Türkçe
  ],
  home: const HomePage(),
);
```

1 `localizationsDelegates`

Çevirileri yükleyen "fabrikalar". Her delegate, seçilen dil için bir çeviri nesnesi üretir.

2 `supportedLocales`

Uygulamanın desteklediği diller. Flutter yalnızca bu listedeki dillere geçer.

3 `Locale('tr')`

Bir dili (ve isteğe bağlı bölgeyi) temsil eden sınıf. Ayrıntısı ileride.

Sonuç (cihaz Türkçe iken):

Material diyalog butonları:

İPTAL TAMAM

Locale: Dil Kodu + Ülke Kodu

Dil kodu · languageCode

ISO 639 · küçük harf

- `tr` Türkçe
- `en` İngilizce
- `de` Almanca
- `ar` Arapça
- `zh` Çince

Ülke / bölge kodu · countryCode

ISO 3166 · büyük harf

- `TR` Türkiye
- `US` ABD
- `GB` Birleşik Krallık
- `DE` Almanya
- `EG` Mısır
- `TW` Tayvan

İsteğe bağlı · scriptCode

`Hant`

Yazı sistemi: zh_Hans basitleştirilmiş, zh_Hant geleneksel Çince

Formül

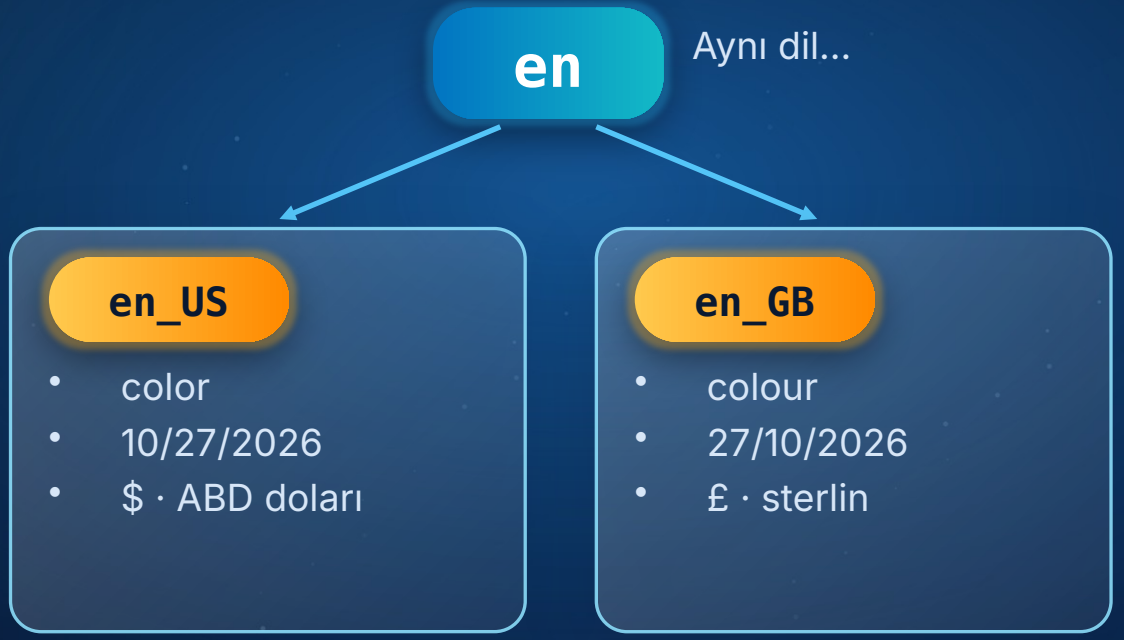
Locale

```
= dil kodu
+ (yazı sistemi)
+ ülke/bölge kodu
```

Sonraki slaytta parçalar birleşiyor →

Locale Örnekleri: Parçalar Birleşiyor

- `tr` _ `TR` — Türkçe · Türkiye
- `en` _ `US` — İngilizce · ABD
- `en` _ `GB` — İngilizce · Birleşik Krallık
- `de` _ `DE` — Almanca · Almanya
- `ar` _ `EG` — Arapça · Mısır
- `zh` _ `Hant` _ `TW` — Çince (geleneksel) · Tayvan



...farklı bölge: yazım, tarih sırası ve para birimi değişir.

locale.dart

```dart
const tr = Locale('tr', 'TR');   // Dil + ülke
const zh = Locale.fromSubtags(languageCode: 'zh', scriptCode: 'Hant', countryCode: 'TW');
```

BÖLÜM

03

ARB Dosyaları ve Çeviri Sistemi

ARB nedir?

app_en.arb · app_tr.arb

Çeviri akışı

l10n.yaml

gen-l10n

AppLocalizations



ARB Dosyaları Nedir?

{ } app_de.arb
Almanca

{ } app_tr.arb
Türkçe

{ } app_en.arb
Şablon (template)

ARB = Application Resource Bundle
JSON tabanlı çeviri dosyası. Her dil için bir dosya; gen-l10n bu dosyalardan Dart kodu üretir.

lib/l10n/app_en.arb

```
1 {
2   "@@locale": "en",
3   "helloWorld": "Hello World!",
4   "@helloWorld": {
5     "description": "The conventional newborn programmer
6 greeting"
7   }
  }
```

Anahtar (key)
"helloWorld" → üretilen Dart getter'ının adı olur

Değer (value)
"Hello World!" → o dildeki çeviri metni

@anahtar
Metadata: açıklama, placeholder tanımları; çevirmene bağlam verir

ARB Dosyalarının Oluşturulması

lib/l10n/app_en.arb

```
{
  "@@locale": "en",
  "appTitle": "Language Demo",
  "helloWorld": "Hello World!",
  "@helloWorld": {
    "description": "Greeting on home
page"
  },
  "changeLanguage": "Change language"
}
```

lib/l10n/app_tr.arb

```
{
  "@@locale": "tr",
  "appTitle": "Dil Demosu",
  "helloWorld": "Merhaba Dünya!",
  "changeLanguage": "Dili değiştir"
}
```

Anahtarlar aynı kalır

Tüm dillerde "helloWorld" aynı; yalnızca değer çevrilir.

Metadata şablonda

@helloWorld gibi açıklamalar app_en.arb (template) dosyasında tutulur.

Dosya adı = dil

app_<locale>.arb: app_en.arb, app_tr.arb, app_de.arb...

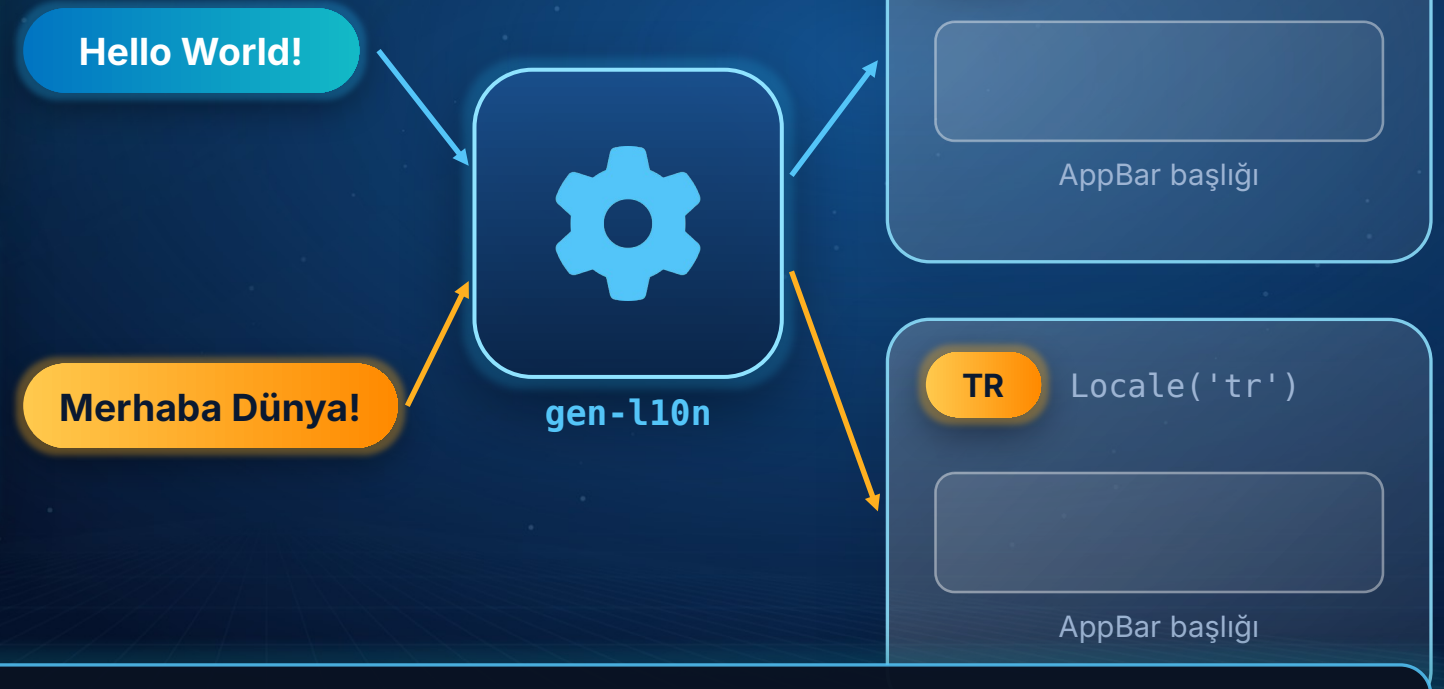
Çeviriler ARB'den Uygulamaya Nasıl Akar?

app_en.arb

```
{
  "helloWorld": "Hello World!"
}
```

app_tr.arb

```
{
  "helloWorld": "Merhaba
Dünya!"
}
```



`AppLocalizations.of(context)!.helloWorld` → aktif Locale 'en' ise "Hello World!", 'tr' ise "Merhaba Dünya!"

l10n.yaml Yapılandırması

l10n.yaml (proje kökü)

```yaml
arb-dir: lib/l10n
template-arb-file: app_en.arb
output-localization-file:
app_localizations.dart
# İsteğe bağlı:
# nullable-getter: false
# untranslated-messages-file:
untranslated.json
```

Üretilen dosyalar (lib/l10n/ içinde)

- app_localizations.dart ← soyut sınıf + delegate
- app_localizations_en.dart ← İngilizce
- app_localizations_tr.dart ← Türkçe

Seçenek	Görevi	Varsayılan
arb-dir	ARB dosyalarının klasörü	lib/l10n
template-arb-file	Kod üretiminde temel alınan ARB	app_en.arb
output-localization-file	Üretilecek Dart dosyasının adı	app_localizations.dart
output-class	Üretilen sınıfın adı	AppLocalizations
nullable-getter	false ise of(context) null döndürmez	true
untranslated-messages-file	Çevrilmemiş mesajları JSON'a yazar	—
preferred-supported-locales	Varsayılan dil sırası, örn. [en_US]	alfabetik
use-escaping	Tek tırnakla kaçış sözdizimi	kapalı

Flutter gen-l10n: Otomatik Kod Üretimi



app_localizations*.dart (sadeleştirilmiş)

```dart
abstract class AppLocalizations {
  static AppLocalizations? of(BuildContext context) =>
      Localizations.of<AppLocalizations>(context, AppLocalizations);
  static const delegate = _AppLocalizationsDelegate();
  String get helloWorld;
}
class AppLocalizationsTr extends AppLocalizations {
  @override
  String get helloWorld => 'Merhaba Dünya!';
}
```

Üretilen sınıf neler sunar?

- of(context)
- delegate
- supportedLocales
- localizationsDelegates
- Her anahtar için getter / metot

Üretilen dosyaları elle düzenlemeyin; ARB'yi değiştirip yeniden üretin.

Uygulamada Çevirilerin Kullanılması

AppLocalizations

Üretilen sınıf

gen-l10n'in ARB'den ürettiği çeviri sınıfı.

.of(context)

Bağlamdan bulma

Widget ağacında yukarı çıkıp o anki dilin çeviri nesnesini bulur.

!

Null kontrolü

of() nullable döner; '!' null olmadığını belirtir (nullable-getter: false ise gerekmez).

.helloWorld

Anahtar → getter

ARB'deki "helloWorld" anahtarı; o dildeki metni döndürür.

home_page.dart

```dart
@override
Widget build(BuildContext context) {
  final l10n = AppLocalizations.of(context)!;
  return Scaffold(
    appBar: AppBar(title: Text(l10n.appTitle)),
    body: Center(child: Text(l10n.helloWorld)),
  );
}
```





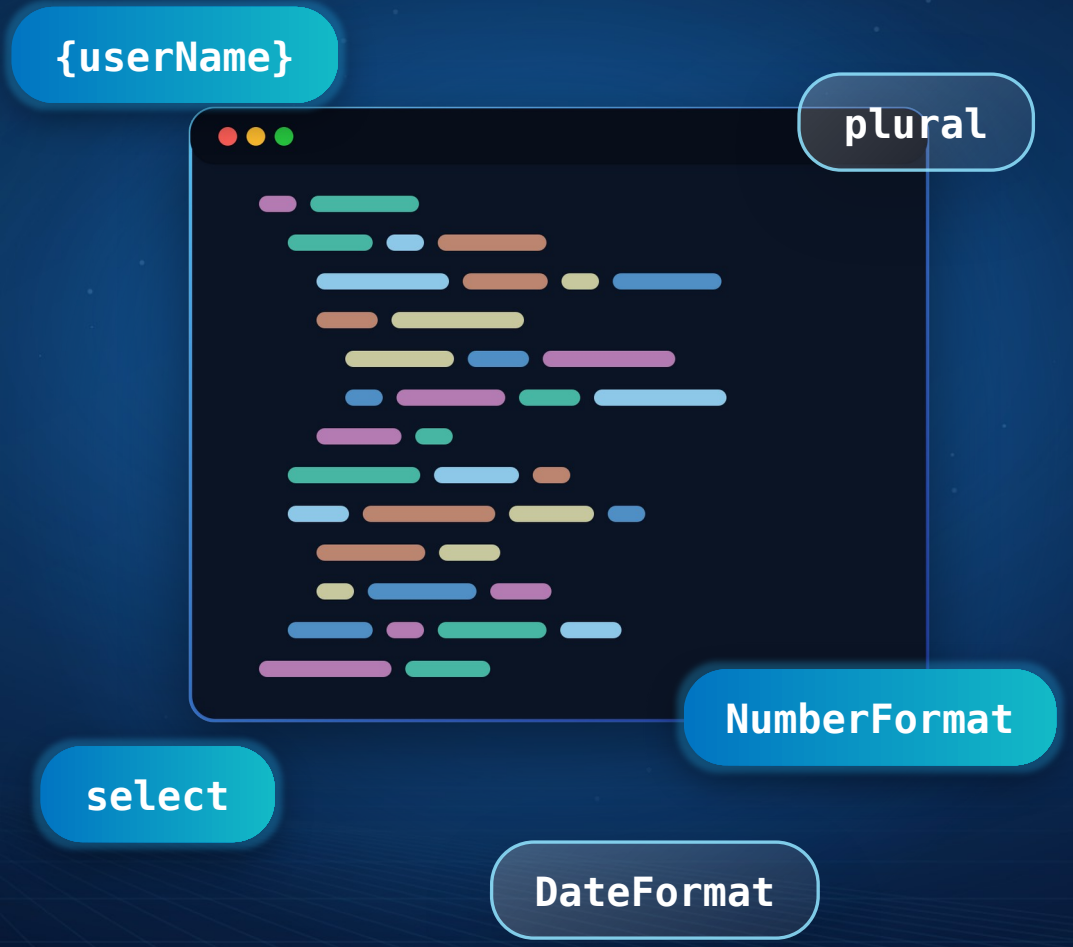

of(context), MaterialApp'in altındaki bir widget'ın context'iyle çağrılmalıdır. Başlık için onGenerateTitle kullanılır.

BÖLÜM

04

Gelişmiş Yerelleştirme

- Placeholder
- Plural
- Select
- Tarih ve saat
- Sayı ve para birimi



Parametreli Çeviriler (Placeholders)

app_en.arb

```
"hello": "Hello {userName}!",
"@hello": {
  "description": "Greeting with the user's name",
  "placeholders": {
    "userName": {
      "type": "String",
      "example": "Ahmet"
    }
  }
}
```

app_tr.arb

```
"hello": "Merhaba {userName}!"
```

home_page.dart

```
Text(l10n.hello('Ahmet'))
```

'Ahmet'

EN **Hello Ahmet!**

TR **Merhaba Ahmet!**

Neden "Hello " + name birleştirmesi değil? Kelime sırası dile göre değişir: `"Welcome back, {name}!"` ↔ `"{name}, tekrar hoş geldin!"`. Placeholder, çevirmene cümleyi kendi dilinin kuralına göre kurma özgürlüğü verir.

Çoğul İfadeler (Pluralization)

app_en.arb

```
"nMessages": "{count, plural, =0{No messages} =1{1 message} other{{count} messages}}",
"@nMessages": { "placeholders": { "count": { "type": "int" } } }
```

app_tr.arb

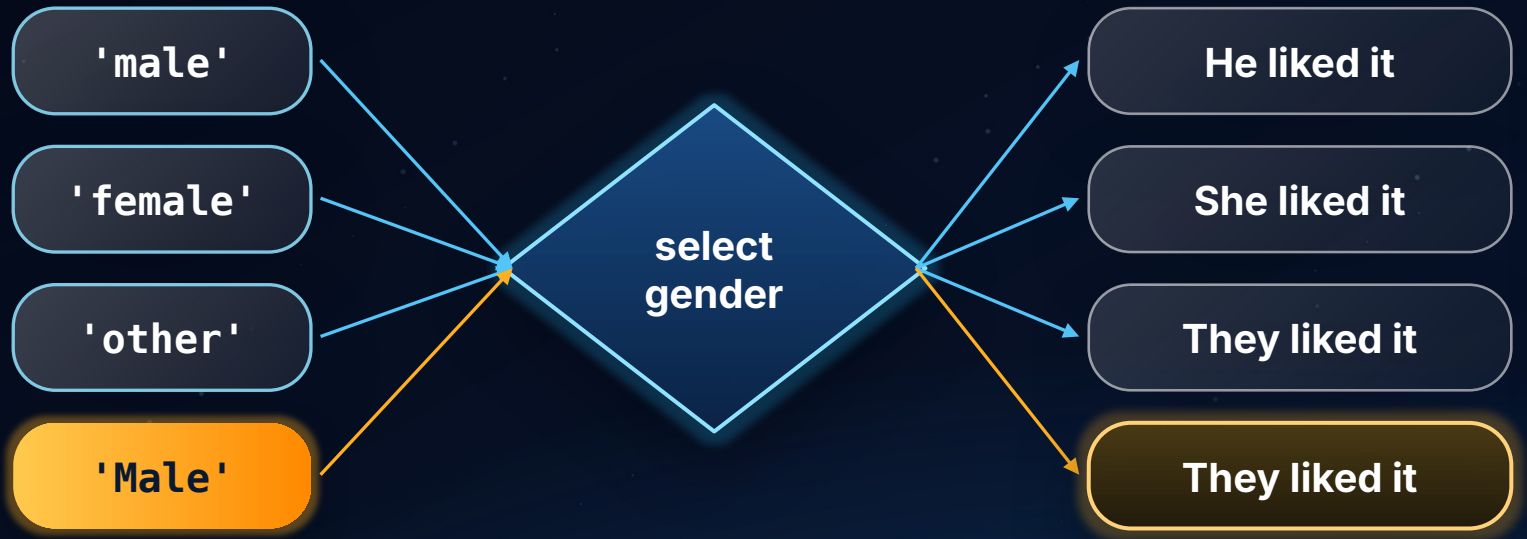
```
"nMessages": "{count, plural, =0{Mesaj yok} =1{1 mesaj} other{{count} mesaj}}"
```

count	English	Türkçe	
0	No messages	Mesaj yok	=0
1	1 message	1 mesaj	=1
5	5 messages	5 mesaj	other

Diller farklı çoğul kategorileri kullanır (Unicode CLDR)

	zero	one	two	few	many	other
Türkçe	zero	one	two	few	many	other
İngilizce	zero	one	two	few	many	other
Rusça	zero	one	two	few	many	other
Arapça	zero	one	two	few	many	other

Select İfadeleri: Değere Göre Metin Seçimi



Büyük/küçük harf duyarlı: 'Male' eşleşmez → other

Kurallar ve ipuçları

- other durumu zorunludur (eşleşmeyen her değer).
- Placeholder tipi String olur.
- Türkçede "o" zamiri cinsiyetsizdir; tüm durumlar aynı metni verebilir.
- Cinsiyet dışında da kullanılır: {plan, select, free{...} premium{...} other{...}}

app_en.arb

```
"liked": "{gender, select, male{He liked it} female{She liked it} other{They liked it}}"
```

app_tr.arb

```
"liked": "{gender, select, male{Beğendi} female{Beğendi} other{Beğendi}}"
```

Tarih ve Saat Yerelleştirmesi

DateTime

27

Ekim 2026

15:30

Locale	yMd	yMMMMd	jm
en_US	10/27/2026	October 27, 2026	3:30 PM
en_GB	27/10/2026	27 October 2026	15:30
tr_TR	**27.10.2026**	**27 Ekim 2026**	**15:30**
de_DE	27.10.2026	27. Oktober 2026	15:30
ja_JP	2026/10/27	2026 年 10 月 27 日	15:30

date_example.dart

```dart
import 'package:intl/intl.dart';
final date = DateTime(2026, 10, 27, 15, 30);
final tag = Localizations.localeOf(context).toString();
DateFormat.yMMMMd(tag).format(date); // 27 Ekim 2026
DateFormat.jm(tag).format(date);     // 15:30
```

ARB'de DateTime placeholder

```
"type": "DateTime", "format": "yMd"
```

Resmi dokümandaki örnek: 9 Temmuz 1959 → ABD İngilizcesi: 7/9/1959, Rusça: 9.07.1959

Sayı ve Para Birimi Biçimlendirme

1234567.89 sayısı:

binlik ayırıcı ondalık ayırıcı *en_IN: Hint gruplaması (2'li basamak)*

en_US	tr_TR	de_DE	fr_FR	en_IN
1,234,567.89	1.234.567,89	1.234.567,89	1 234 567,89	12,34,567.89

simpleCurrency ile para birimi:

en_US	tr_TR	de_DE	ja_JP
$1,234,567.89	₺1.234.567,89	1.234.567,89 €	¥ 1,234,568

number_example.dart

```dart
final n = 1234567.89;
NumberFormat.decimalPattern('tr_TR').format(n);          // 1.234.567,89
NumberFormat.simpleCurrency(locale: 'tr_TR').format(n); //
₺1.234.567,89
NumberFormat.compact(locale: 'en_US').format(1200000);  // 1.2M
```

ARB "format"	1200000 (en)
compact	1.2M
compactLong	1.2 million
decimalPattern	1,200,000
simpleCurrency	$1,200,000

BÖLÜM

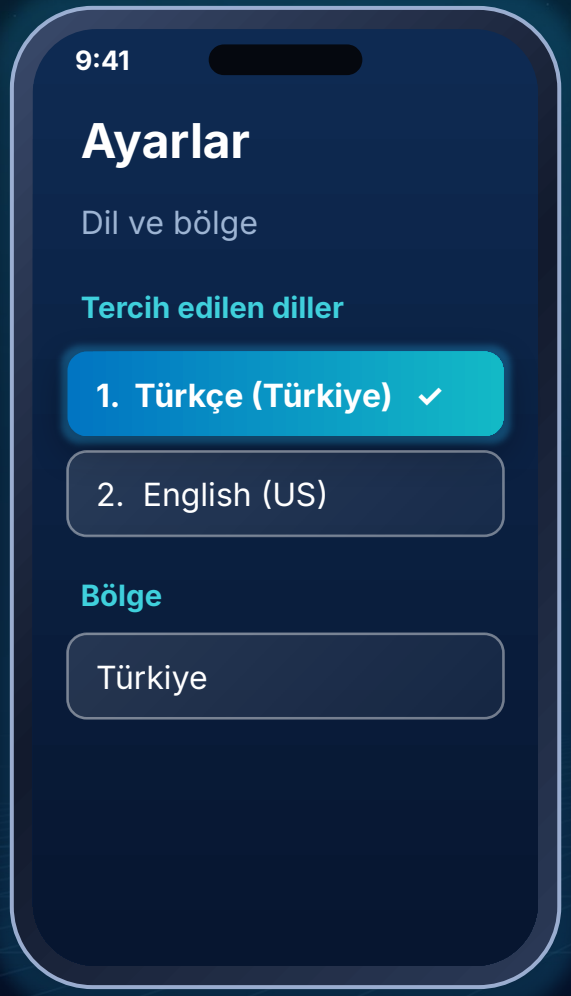
05

Dil Yönetimi ve Kullanıcı Deneyimi

- Cihaz dili
- Uygulama içinden dil değiştirme
- Fallback
- RTL / LTR



Cihaz Dilinin Algılanması



örnek ayarlar ekranı

1 **İşletim sistemi tercih edilen dil listesini verir**

örn. [tr_TR, en_US]

↓

2 **Flutter listeyi supportedLocales ile eşleştirir**

Eşleşme yoksa fallback kuralları (ileride)

↓

3 **Sonuç Locale, Localizations widget'ına verilir**

Localizations.localeOf(context) ile okunur

↓

4 **Kullanıcı cihaz dilini değiştirirse**

Uygulama yeni dille otomatik yeniden çizilir

locale_info.dart

```dart
// Cihazın dil tercih listesi
final device =
    WidgetsBinding.instance
        .platformDispatcher.local
es;

// Uygulamanın kullandığı dil
final current =
    Localizations.localeOf(context);
```

Sonuç

`tr_TR → Locale('tr')`

supportedLocales'ta yalnızca 'tr' olduğu için dil koduyla eşleşir.

Uygulama İçinden Dil Değiştirme

main.dart

```dart
final appLocale = ValueNotifier<Locale?>(null); // null → cihaz dili

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return ValueListenableBuilder<Locale?>(
      valueListenable: appLocale,
      builder: (context, locale, _) => MaterialApp(
        locale: locale, // seçilen dil
        localizationsDelegates:
AppLocalizations.localizationsDelegates,
        supportedLocales: AppLocalizations.supportedLocales,
        home: const HomePage(),
      ),
    );
  }
}
// Dil butonunda: appLocale.value = const Locale('tr');
```



EN'e dokun → arayüz İngilizceye döner

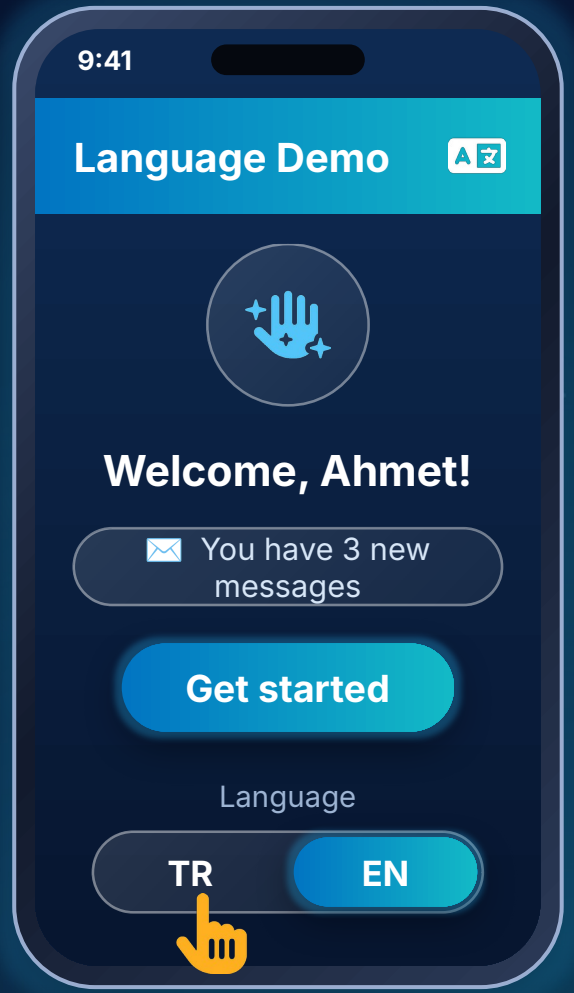
Uygulama İçinden Dil Değiştirme: English Seçildi

main.dart

```dart
final appLocale = ValueNotifier<Locale?>(null); // null → cihaz dili

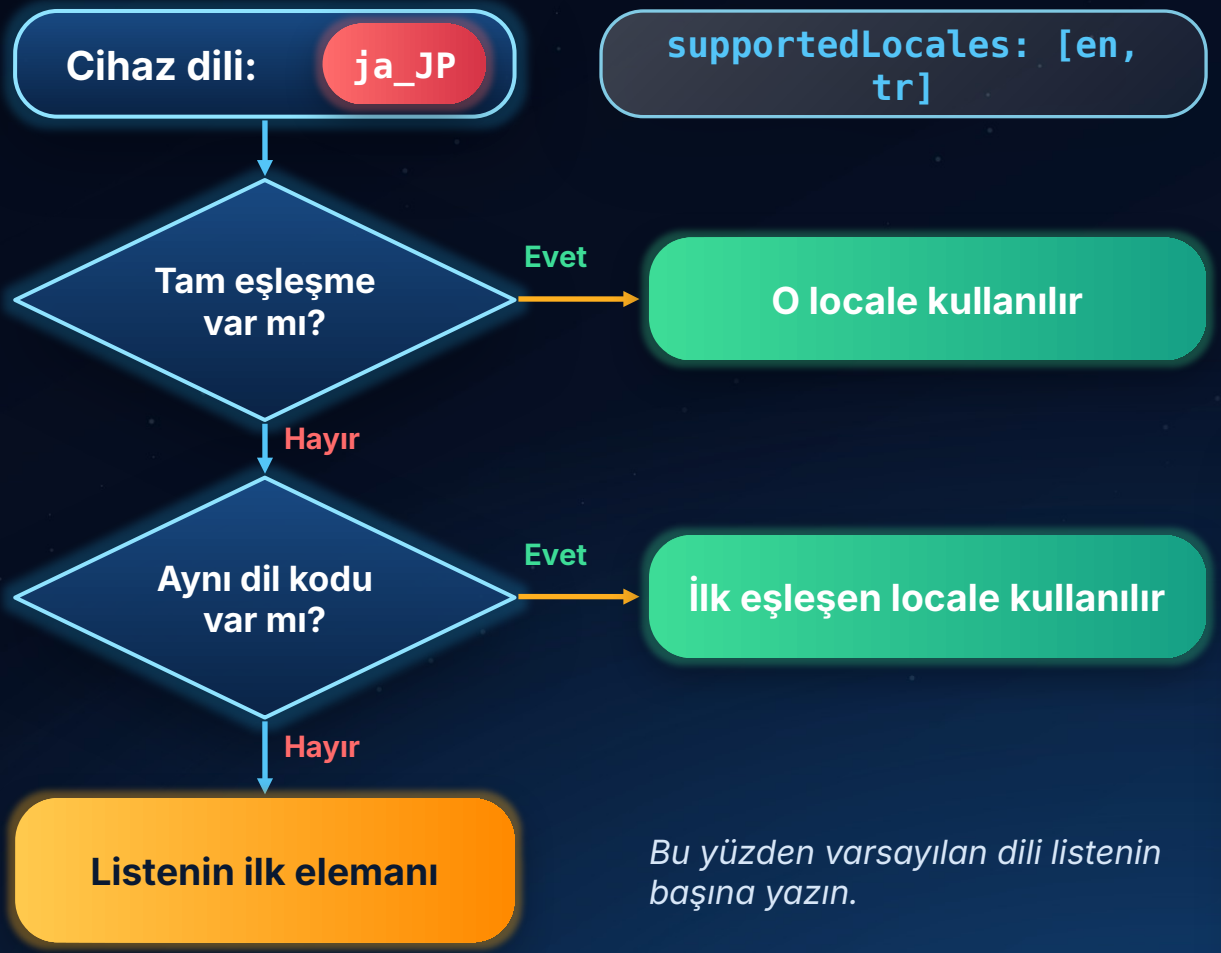
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return ValueListenableBuilder<Locale?>(
      valueListenable: appLocale,
      builder: (context, locale, _) => MaterialApp(
        locale: locale, // seçilen dil
        localizationsDelegates:
AppLocalizations.localizationsDelegates,
        supportedLocales: AppLocalizations.supportedLocales,
        home: const HomePage(),
      ),
    );
  }
}
// Dil butonunda: appLocale.value = const Locale('tr');
```



TR'ye dokununca tekrar Türkçe olur

Desteklenmeyen Diller ve Fallback



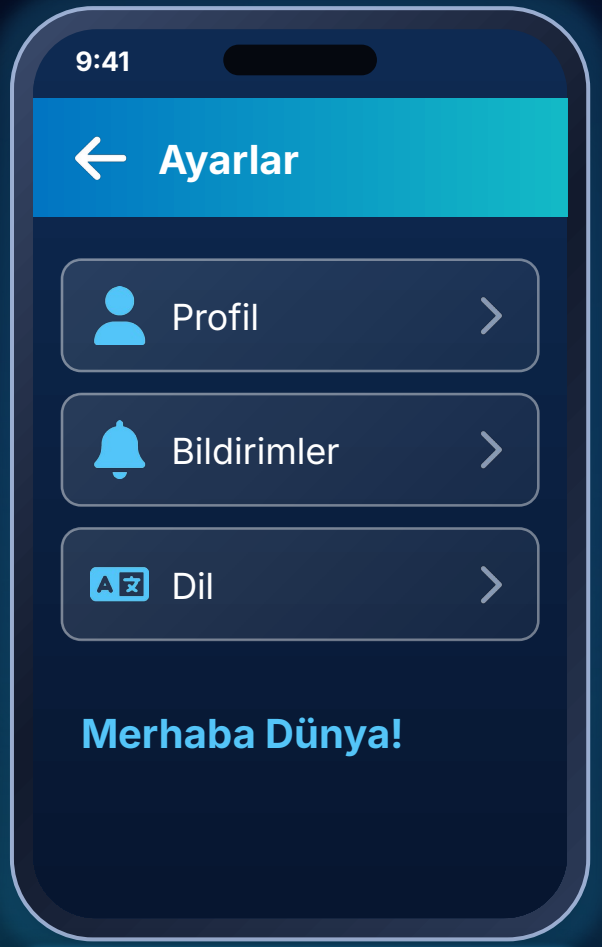
Bu yüzden varsayılan dili listenin başına yazın.

Cihaz	Sonuç	Neden?
tr_TR	**tr**	dil kodu eşleşti
en_GB	**en**	dil kodu eşleşti
en	**en**	tam eşleşme
ja_JP	**en**	ilk eleman (fallback)

main.dart

```dart
MaterialApp(
  localeResolutionCallback: (locale, supported)
{
    for (final s in supported) {
      if (s.languageCode ==
locale?.languageCode) {
        return s;
      }
    }
    return supported.first; // varsayılan
  },
)
```

LTR ve RTL Desteği



LTR · Türkçe

örnek arayüz

Okuma yönü

soldan sağa →

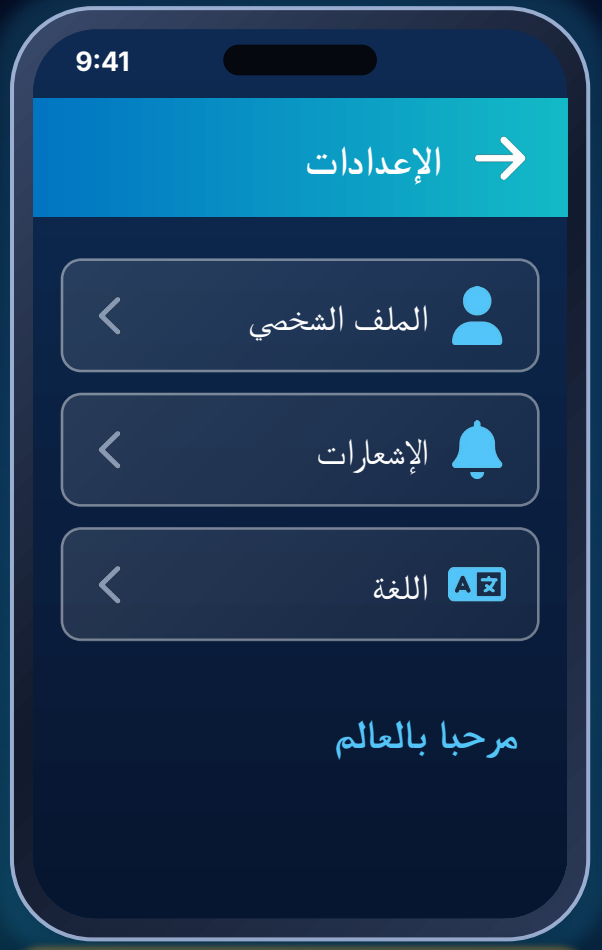
`GlobalWidgetsLocalizations`

delegate'i, locale'e göre metin yönünü (TextDirection) ayarlar.

`Directionality.of(context)`

ile o anki yön (ltr / rtl) okunabilir.

RTL: Sağdan Sola Yerleşim



RTL · Arapça (العربية) örnek arayüz

Okuma yönü

sağdan sola ←

left / right yerine start / end

EdgeInsetsDirectional, AlignmentDirectional, TextAlign.start kullanın.

Otomatik aynalama

Row, ListTile, AppBar gibi widget'lar RTL'de sağdan sola dizilir.

BÖLÜM

06

Uygulamalı Örnek

- Dil Demosu uygulaması
- Kod yapısı
- Çalışma mantığı
- TR / EN karşılaştırma



Örnek Uygulama: "Dil Demosu"



Tasarım amaçlı örnek arayüz

ARB anahtarı	Türkçe (app_tr.arb)	English (app_en.arb)
`appTitle`	Dil Demosu	Language Demo
`welcome(name)`	Hoş geldin, {name}!	Welcome, {name}!
`nMessages(count)`	{count} yeni mesajın var	You have {count} new messages
`startButton`	Başla	Get started
`languageLabel`	Dil	Language

Kapsanan konular: sabit metin · placeholder · plural · buton · dil seçimi

Uygulamanın Kod Yapısı

EXPLORER · dil_demo

```
▾ dil_demo/
    pubspec.yaml                    generate: true
    l10n.yaml                       gen-l10n ayarları
  ▾ lib/
      main.dart                     MaterialApp + locale
      home_page.dart                arayüz
      language_switch.dart          dil seçici
    ▾ l10n/
        app_en.arb                  şablon
        app_tr.arb                  Türkçe
        app_localizations.dart      ⚙ üretilir
app_localizations_en.dart           ⚙ üretilir
app_localizations_tr.dart           ⚙ üretilir
```

lib/home_page.dart

```dart
class HomePage extends StatelessWidget {
  const HomePage({super.key});
  @override
  Widget build(BuildContext context) {
    final l10n = AppLocalizations.of(context)!;
    return Scaffold(
      appBar: AppBar(title: Text(l10n.appTitle)),
      body: Column(
        children: [
          Text(l10n.welcome('Ahmet')),
          Text(l10n.nMessages(3)),
          FilledButton(
            onPressed: () {},
            child: Text(l10n.startButton),
          ),
          const LanguageSwitch(),
        ],
      ),
    );
  }
}
```

Kullanıcı Dil Seçtiğinde Arka Planda Neler Olur?

1. Kullanıcı "EN" segmentine dokunur
2. onSelectionChanged → appLocale.value = Locale('en')
3. ValueListenableBuilder tetiklenir; MaterialApp yeni locale ile kurulur
4. Localizations, AppLocalizations.delegate.load(Locale('en')) çağırır
5. AppLocalizationsEn gelir; of(context) kullanan widget'lar yeniden çizilir
6. Ekranda: "Language Demo", "Welcome, Ahmet!"

lib/language_switch.dart

```dart
class LanguageSwitch extends StatelessWidget {
  const LanguageSwitch({super.key});

  @override
  Widget build(BuildContext context) {
    final code =
Localizations.localeOf(context).languageCode;
    return SegmentedButton<String>(
      segments: const [
        ButtonSegment(value: 'tr', label:
Text('TR')),
        ButtonSegment(value: 'en', label:
Text('EN')),
      ],
      selected: {code},
      onSelectionChanged: (selection) {
        appLocale.value = Locale(selection.first);
      },
    );
  }
}
```

Türkçe ve İngilizce Ekran Karşılaştırması



Türkçe · tr

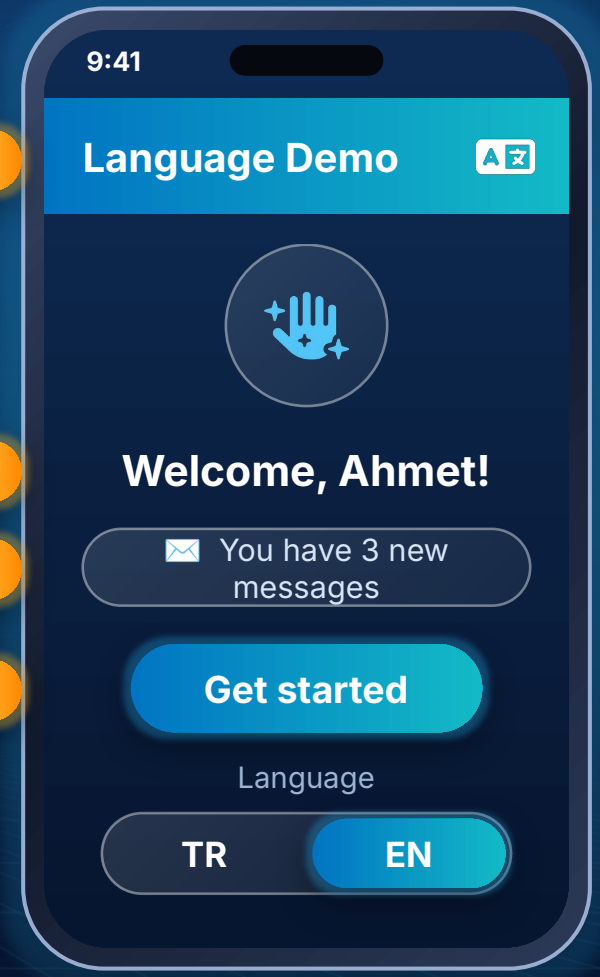
1 **Başlık**
Dil Demosu ↔ Language Demo

2 **Placeholder**
{name} aynı, cümle yapısı farklı

3 **Çoğul**
"3 yeni mesajın var" ↔ "You have 3 new messages"

4 **Buton**
Başla ↔ Get started (farklı uzunluk!)

Kod aynı — yalnızca Locale değişti. Tasarım amaçlı örnek arayüz.



English · en

BÖLÜM

07

Sonuç



Sık yapılan hatalar

Genel değerlendirme

Kaynakça

✓ i18n altyapısı

✓ ARB + gen-l10n

✓ Locale yönetimi

Sık Yapılan Hatalar ve Çözümleri

Material widget'ları İngilizce kalıyor

GlobalMaterialLocalizations (ve Widgets/Cupertino) delegate'lerini ekleyin.

AppLocalizations.of(context) null

MaterialApp'in altındaki bir context kullanın; başlık için onGenerateTitle.

app_localizations.dart bulunamıyor

generate: true + l10n.yaml; ardından flutter pub get veya flutter gen-l10n.

Bazı metinler çevrilmemiş

Anahtarları tüm ARB'lere ekleyin; untranslated-messages-file ile listeleyin.

iOS'ta diller App Store'da görünmüyor

Xcode → Runner → Info → Localizations bölümüne dilleri ekleyin.

Metin birleştirme ve left/right

Placeholder/plural kullanın; RTL için start/end ve Directional sınıfları.

Genel Değerlendirme: i18n Yol Haritası



Teşekkürler

Sorular?



Flutter ve Dart logoları Google LLC'nin ticari markalarıdır; eğitim amacıyla kullanılmıştır.

Kaynakça

Ana kaynak · Flutter Docs
Internationalizing Flutter apps — docs.flutter.dev/ui/internationalization

Flutter API
flutter_localizations kütüphanesi — api.flutter.dev/flutter/flutter_localizations

Flutter API
Localizations, Locale, MaterialApp.locale — api.flutter.dev

Dart paketi
intl (DateFormat, NumberFormat) — pub.dev/packages/intl

ARB biçimi
Application Resource Bundle — github.com/google/app-resource-bundle

Örnek kodlar
flutter/website · examples/internationalization — github.com/flutter/website

Çoğul kuralları
Unicode CLDR Language Plural Rules — cldr.unicode.org

Erişim tarihi: 27.10.2026